



COURSE CODE:- CSE2006

COURSE NAME: Programming in Java

Fall Semester 2026-27

PROJECT REPORT FILE

**Subject - Bank Management System
(ATM Simulator)**

SUBMITTED BY:-

Anushka Singh

25BAI10936

2. INTRODUCTION

The Automated Teller Machine (ATM) Simulator is a desktop-based banking application developed in Java. It provides a real-time interactive graphical user interface (GUI) mimicking traditional physical ATM kiosks. The system enables users to complete self-service banking operations including account registration, authentication, deposits, withdrawals, fast cash transactions, mini-statements, balance inquiries, and PIN changes.

3. PROBLEM STATEMENT

Traditional banking operations require physical branch visits for basic transactions like cash deposits, withdrawals, and balance queries. This results in long waiting times and operational overhead. This project addresses the need for a standalone, secure Java application that simulates self-service ATM transactions with real-time persistent data management via database connectivity

4. FUNCTIONAL REQUIREMENTS

- **FR-1 User Onboarding:** 3-tier registration (Signup, Signup2, Signup3) to record personal, demographic, and account details.
- **FR-2 Authentication:** Secure login system (Login.java) validating Card Number and PIN.
- **FR-4 Dashboard Navigation:** Transaction selection portal (Transactions.java).
- **FR-4 Deposit Funds:** Cash deposit processing with database ledger update (Deposit.java).
- **FR-5 Cash Withdrawal:** Standard (Withdrawl.java) and quick pre-set (FastCash.java) cash debits with account balance validation.
- **FR-6 Balance Inquiry:** Real-time query of available balance (BalanceEquiry.java).
- **FR-7 Mini Statement:** Display recent account transaction history (MiniStatement.java).
- **FR-9Security Management:** PIN modification interface (Pin.java)

5. NON-FUNCTIONAL REQUIREMENTS

- **Security:** Password field masking for PIN entries and parameterized query checks.
- **Performance:** Real-time transaction response time under 2 seconds.
- **Usability:** High-contrast frame layout emulating standard ATM screen display buttons.
- **Reliability:** Persistent database storage ensuring data consistency across sessions.

System Architecture

The system follows a classic three-tier architecture.

Separating presentation, business logic and data access keeps each concern independent: the console screens can be replaced by a GUI, or MySQL by another RDBMS, without rewriting the transaction rules.

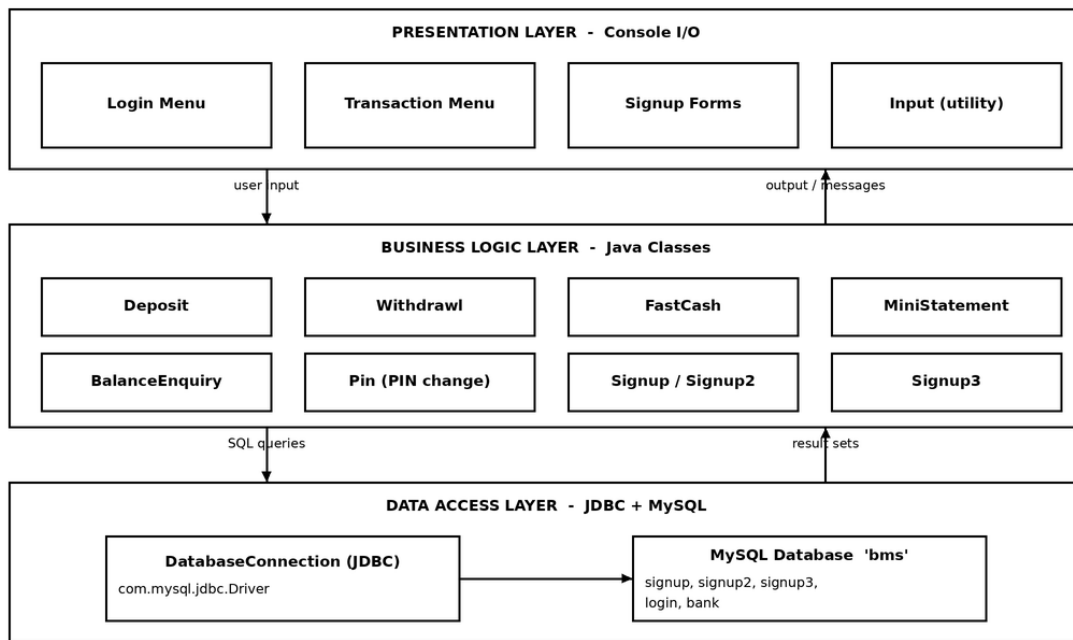
1 Presentation Layer This layer consists of the console menus printed by Login and Transactions and the registration forms of the Signup classes. All keyboard input passes through the Input utility class, which wraps a `BufferedReader` and offers `readLine()` and `readInt()`; `readInt()` keeps re-prompting until a

valid integer is supplied, so numeric parsing errors never reach the layers below.

2 Business Logic Layer Each ATM operation is implemented by its own class - Deposit, Withdrawl, FastCash, MiniStatement, BalanceEnquiry and Pin - constructed with the PIN of the logged-in account and exposing a single start() method. This layer owns all the rules of the system: an amount must be positive, a withdrawal is refused when the computed balance is smaller than the requested amount, and a PIN change is applied only when both entries match and are non-empty.

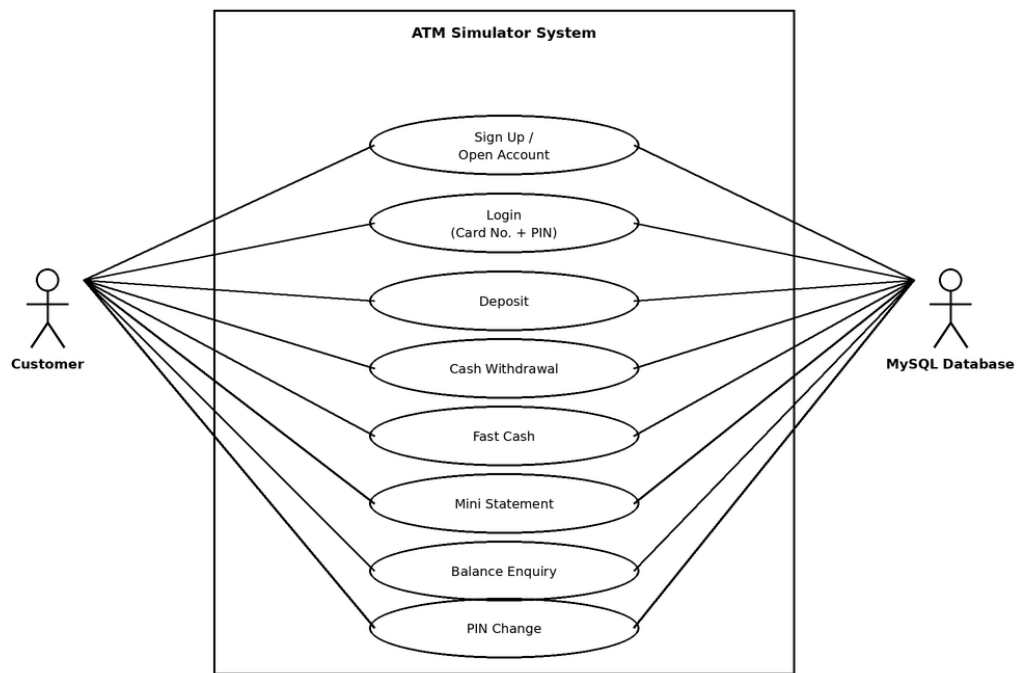
3 Data Access Layer The DatabaseConnection class (Conn.java) loads the MySQL JDBC driver, opens a connection to the schema bms and exposes a Statement object. Every business class creates its own DatabaseConnection instance when it needs to read or write, executing SELECT queries through executeQuery() and INSERT/UPDATE statements through executeUpdate().

ATM Simulator - System Architecture (3-Tier)



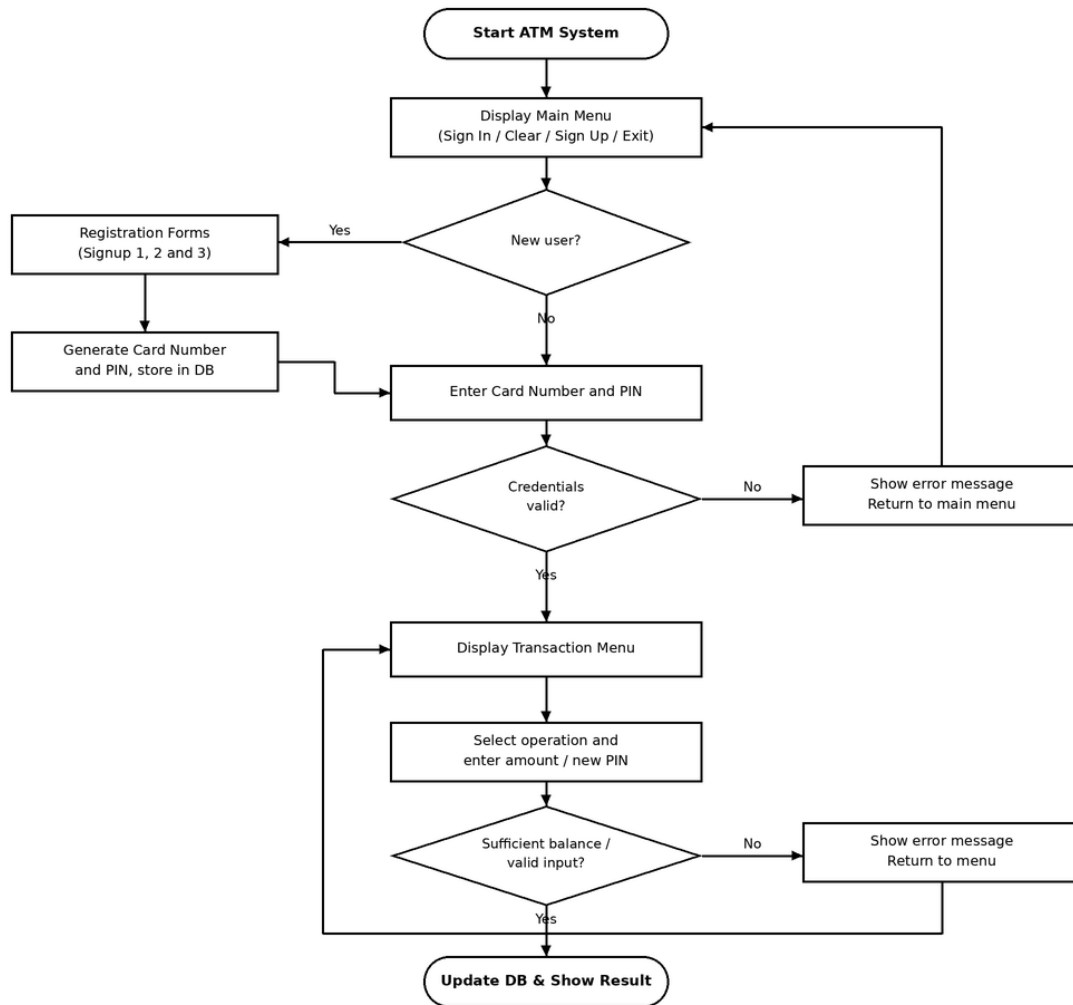
USE CASE DIAGRAM

ATM Simulator System - Use Case Diagram



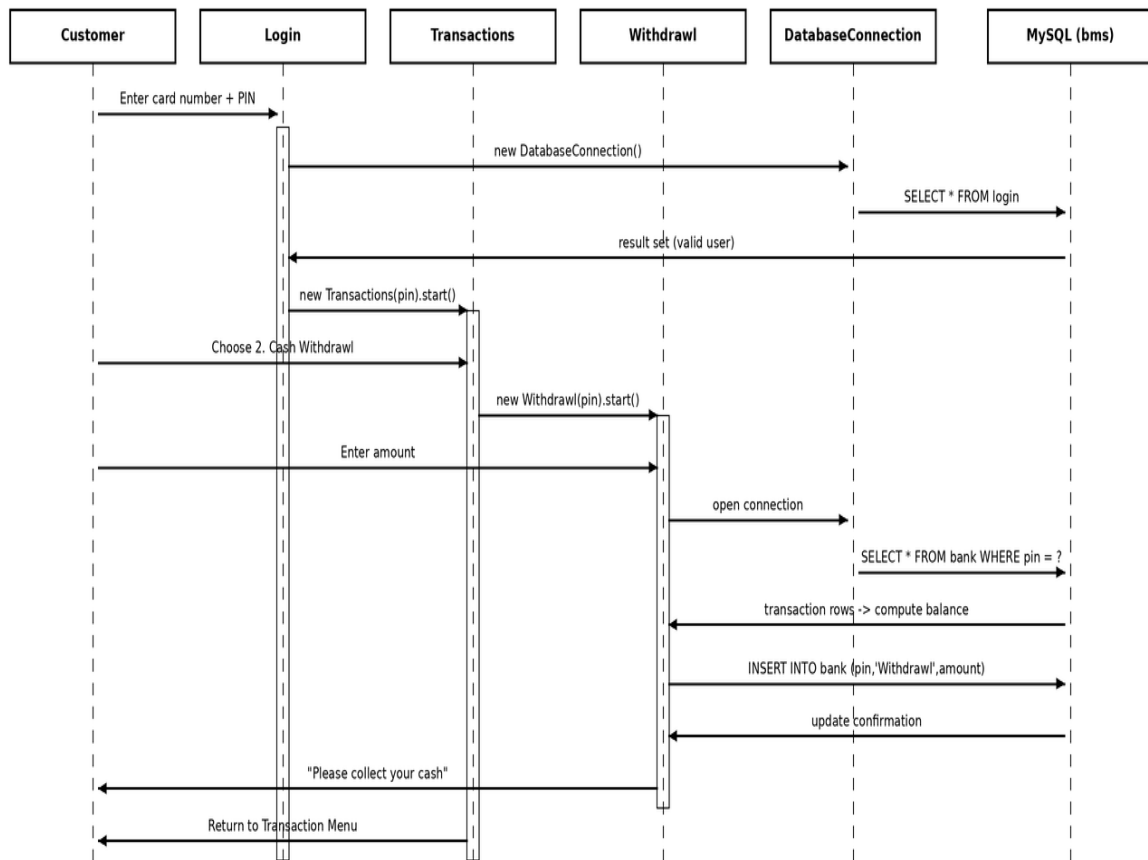
WORKFLOW

ATM Simulator - Workflow Diagram



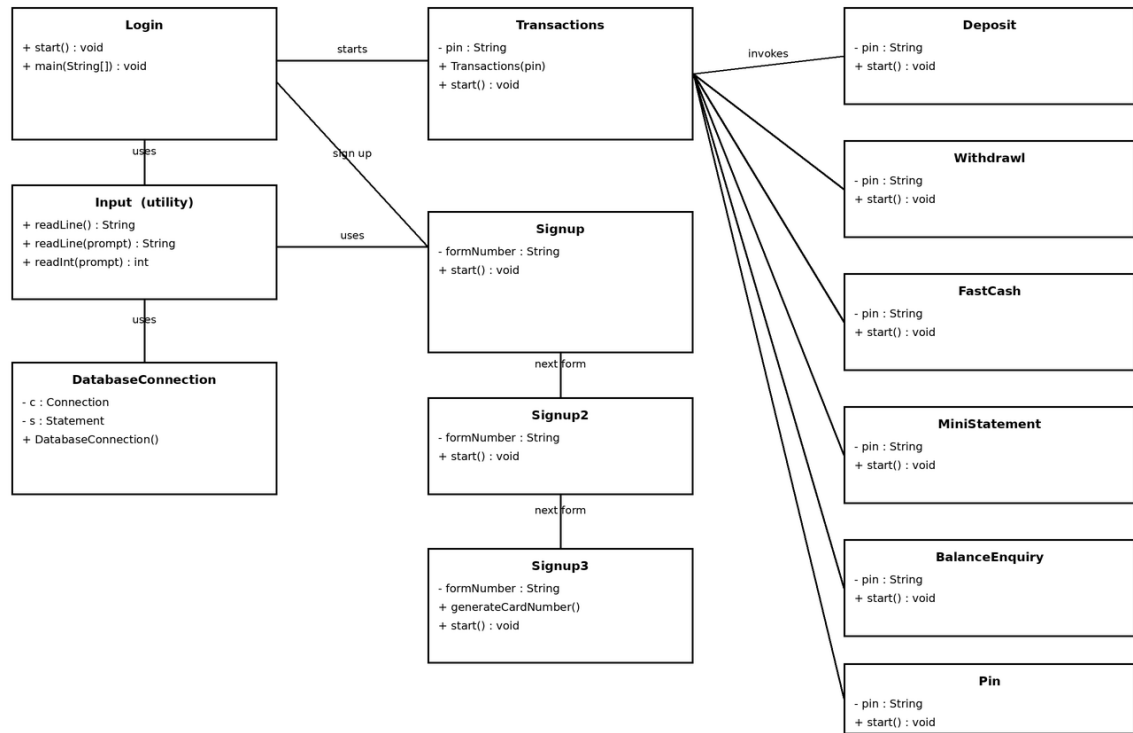
SEQUENCE DIAGRAM

ATM Simulator - Sequence Diagram (Cash Withdrawal)



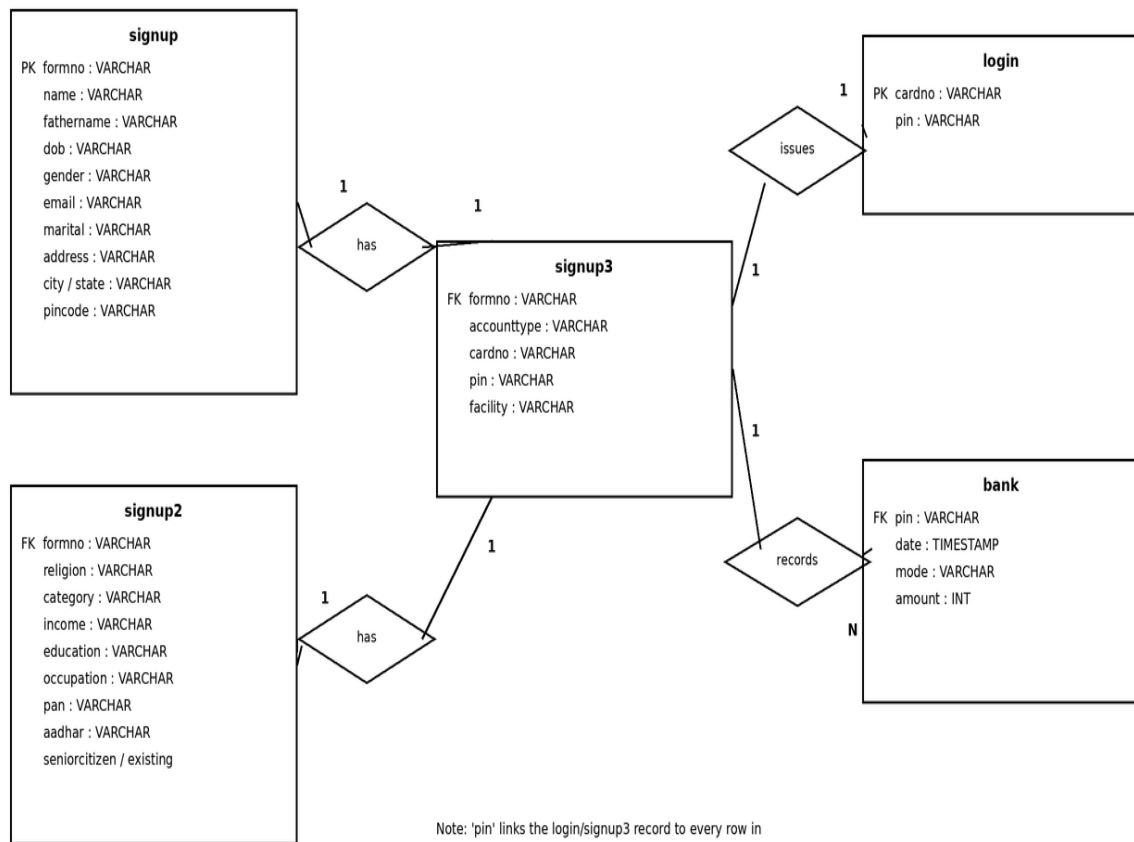
COMPONENT DIAGRAM

ATM Simulator - Class Diagram



ER DIAGRAM

ATM Simulator - ER Diagram (MySQL schema 'bms')



8. Implementation Details

8.1 Technology Stack

Component	Technology Used
Programming Language	Java SE (JDK 8 or above)
Database	MySQL 5.7 / 8.x, schema name <i>bms</i>
Connectivity	JDBC (com.mysql.jdbc.Driver / MySQL Connector-J)
Interface	Command-line console (System.in / System.out)
Package	ASimulatorSystem
Development Tools	VS Code / IntelliJ IDEA, MySQL Workbench, javac and java

8.2 Module Description

Class / File	Responsibility
Login.java	Entry point of the application. Displays the main menu, reads the card number and PIN, validates them against the login table and starts a transaction session. Also contains the Input utility class.
Conn.java	Defines DatabaseConnection, which loads the JDBC driver, connects to the bms schema and creates the Statement used by all other classes.
Signup.java	Collects and stores personal details against a form number, then invokes Signup2.
Signup2.java	Collects and stores additional details such as PAN, Aadhar, income and occupation, then invokes Signup3.
Signup3.java	Collects account type, services and declaration; generates the card number and PIN using java.util.Random, inserts records into signup3 and login and starts an initial deposit.
Transactions.java	Displays the transaction menu and delegates the selected option to the corresponding operation class.
Deposit.java	Validates that the amount is positive and inserts a Deposit row into the bank table.
Withdrawl.java	Computes the current balance, refuses the request when funds are insufficient and otherwise inserts a Withdrawl row.
FastCash.java	Offers fixed denominations and performs the same balance check and insertion as a normal withdrawal.
MiniStatement.java	Retrieves all rows for the account, prints them as a formatted table and shows the closing balance.
BalanceEquiry.java	Defines BalanceEnquiry, which aggregates the transaction rows and displays the current balance.
Pin.java	Validates and applies a PIN change across the bank, login and signup3 tables.

. Design Decisions and Rationale

1 Console interface instead of a GUI A text-based interface was chosen so that the project could concentrate on transaction logic and database interaction rather than on layout code. It also makes the application runnable on any machine with a JDK, including headless environments, and keeps the source small enough to be reviewed in full.

.2 One class per transaction Deposit, Withdrawl, FastCash, MiniStatement, BalanceEnquiry and Pin are separate classes with an identical public shape. The alternative - a single large class with a switch statement - would have concentrated unrelated rules in one place. With the chosen design a defect in withdrawal logic cannot affect the mini statement, and a new operation requires only a new class and one extra menu entry in Transactions.

3 Balance derived, never stored No column holds the account balance. Both BalanceEnquiry and Withdrawl iterate over the transaction rows of the account and add deposits while subtracting withdrawals. Storing a balance column would introduce the risk of the stored value disagreeing with the transaction history after a partial failure; deriving it guarantees that the two can never diverge.

4 PIN as the transaction key Once a user is authenticated, the PIN is passed to each operation class and used to filter the bank table. This keeps the constructor of every operation

identical. Because the PIN is the link between the login record and the transaction history, the PIN change operation updates the value in the bank, login and signup3 tables within the same routine so the account remains consistent.

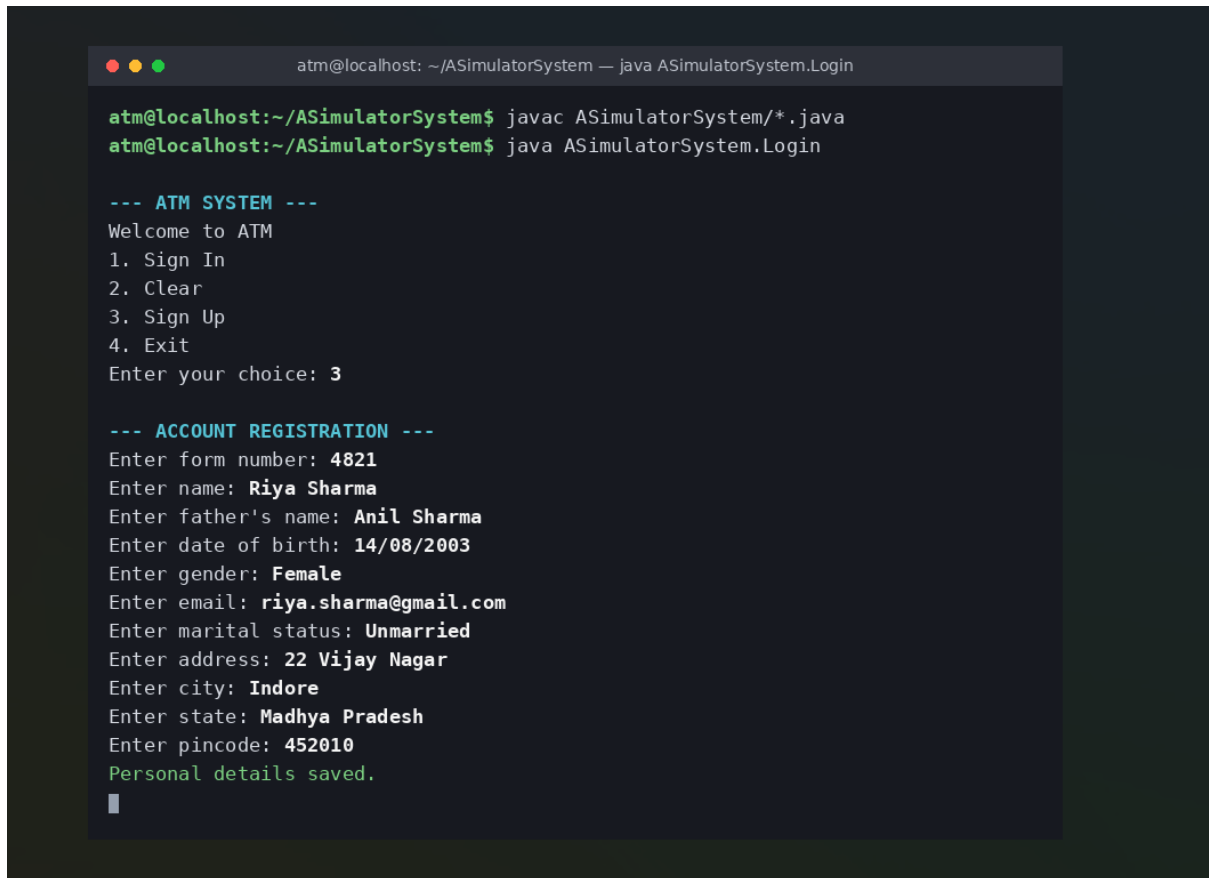
5 Three-stage registration Account opening is split into personal details, additional details and account details, mirroring the multi-page forms used by real banks. The form number entered in the first stage is carried through the chain and acts as the key that joins the three registration tables.

.6 Separate connection object per operation Each operation creates its own DatabaseConnection. This is simple and avoids passing a shared connection through every constructor, at the cost of repeated connection setup. A connection pool or a single shared connection would be the natural optimisation in a production system.

7 Known design limitations

- SQL statements are built by string concatenation, which makes the application vulnerable to SQL injection; PreparedStatement is the correct remedy and is listed as future work.
- The PIN is stored and compared in plain text; a salted hash such as BCrypt would be used in a real deployment.
- Database credentials are hard-coded in Conn.java and should be externalised into a properties file.

SCREENSHOTS



```
atm@localhost: ~/ASimulatorSystem — java ASimulatorSystem.Login

atm@localhost:~/ASimulatorSystem$ javac ASimulatorSystem/*.java
atm@localhost:~/ASimulatorSystem$ java ASimulatorSystem.Login

--- ATM SYSTEM ---
Welcome to ATM
1. Sign In
2. Clear
3. Sign Up
4. Exit
Enter your choice: 3

--- ACCOUNT REGISTRATION ---
Enter form number: 4821
Enter name: Riya Sharma
Enter father's name: Anil Sharma
Enter date of birth: 14/08/2003
Enter gender: Female
Enter email: riya.sharma@gmail.com
Enter marital status: Unmarried
Enter address: 22 Vijay Nagar
Enter city: Indore
Enter state: Madhya Pradesh
Enter pincode: 452010
Personal details saved.
█
```

atm@localhost: ~/ASimulatorSystem — java ASimulatorSystem.Login

--- **ADDITIONAL DETAILS** ---

Enter religion: **Hindu**
Enter category: **General**
Enter income: **Below 2,50,000**
Enter education qualification: **Graduate**
Enter occupation: **Student**
Enter PAN number: **BKHPS4471L**
Enter Aadhar number: **6721 4498 3025**
Are you a senior citizen? (Yes/No): **No**
Do you have an existing account? (Yes/No): **No**
Additional details saved.

--- **ACCOUNT DETAILS** ---

Enter account type (Saving/Fixed Deposit/Current/Recurring): **Saving**
Enter services required (ATM Card/Internet Banking/Mobile Banking): **ATM Card**
Do you agree with the declaration? (Yes/No): **Yes**

Account created successfully!
Your Card Number: 5040936217884413
Your PIN: 4417
Please keep these details safe.

■



--- ATM SYSTEM ---

Welcome to ATM

1. Sign In
2. Clear
3. Sign Up
4. Exit

Enter your choice: **1**

Enter Card Number: **5040936217884413**

Enter PIN: **4417**

Login successful.

--- TRANSACTIONS ---

1. Deposit
2. Cash Withdrawl
3. Fast Cash
4. Mini Statement
5. PIN Change
6. Balance Enquiry
7. Exit

Enter your choice: **1**

--- DEPOSIT ---

Enter amount to deposit: **15000**

Rs. 15000 deposited successfully.

--- CASH WITHDRAWAL ---

Enter amount to withdraw: **25000**

Insufficient balance.

--- CASH WITHDRAWAL ---

Enter amount to withdraw: **3500**

Please collect your cash.

Rs. 3500 withdrawn successfully.



```
atm@localhost: ~/ASimulatorSystem — java ASimulatorSystem.Login

--- FAST CASH ---
1. Rs. 100
2. Rs. 500
3. Rs. 1000
4. Rs. 2000
5. Rs. 5000
6. Rs. 10000
7. Back
Enter your choice: 4
Please collect your cash.
Rs. 2000 withdrawn successfully.

--- BALANCE ENQUIRY ---
Your current balance is: Rs. 9500

--- PIN CHANGE ---
Enter new PIN: 7726
Confirm new PIN: 7762
PINs do not match.

--- PIN CHANGE ---
Enter new PIN: 7726
Confirm new PIN: 7726
PIN changed successfully.
```



--- TRANSACTIONS ---

1. Deposit
 2. Cash Withdrawl
 3. Fast Cash
 4. Mini Statement
 5. PIN Change
 6. Balance Enquiry
 7. Exit
- Enter your choice: 4

--- MINI STATEMENT ---

Date	Mode	Amount

2026-09-12	Deposit	Rs. 15000
2026-09-14	Withdrawl	Rs. 3500
2026-09-16	Withdrawl	Rs. 2000
2026-09-18	Deposit	Rs. 2500
2026-09-18	Withdrawl	Rs. 2500

Available Balance: Rs. 9500

Enter your choice: 7
Returning to login...

Enter your choice: 4
Thank you for using the ATM.
atm@localhost:~/ASimulatorSystem\$



Testing Approach Testing was performed manually using a black-box approach at the level of individual operations, followed by integration testing of complete user journeys. For every operation, one valid case and at least one boundary or invalid case were executed, and the effect on the database was verified with SQL queries in MySQL Workbench.

.1 Levels of Testing • Unit level: each operation class was executed in isolation with valid and invalid inputs. • Integration level: the registration chain (Signup to Signup2 to Signup3 to Deposit) and the login-to-transaction flow were tested end to end. • Database verification: after each transaction, the bank table was queried directly to confirm that the row was inserted with the correct mode and amount. • Regression check: after the PIN change, all six transactions were repeated with the new PIN to confirm that the transaction history remained linked to the account

Challenges Faced

1 JDBC driver and connection issues The first attempts to run the application failed with a `ClassNotFoundException` because the MySQL Connector-J JAR was not on the classpath, and later with a connection refused error when the MySQL service was not running. The problem was resolved by adding the connector JAR to the classpath at compile and run time and by confirming the schema name, user and password used in the connection URL. A catch block was then added so that a connection failure reports a readable message instead of only a stack trace.

.2 Keeping the balance consistent An early version stored a balance column that was updated after every transaction. When a withdrawal failed midway, the stored balance no longer matched the transaction rows. The design was changed so that the balance is always recomputed from the bank table, which removed the possibility of inconsistency entirely at the cost of one extra query per operation.

3 Linking the registration stages Splitting registration across three classes required the form number to be carried from one stage to the next. Passing it through each constructor solved the problem, but it also made it clear that an incomplete registration leaves orphan rows in `signup` and `signup2`; validating the chain and cleaning up incomplete forms is noted as future work.

4 Handling invalid user input Parsing the amount directly with Integer.parseInt caused a NumberFormatException whenever the user typed a letter or pressed Enter on an empty line. Centralising input in the Input class, with readInt() looping until a valid number is supplied, removed this class of crash from every operation at once.

5 PIN change across multiple tables Because the PIN links the login record to the transaction history, changing it in only one table would have detached the account from its transactions. The update had to be applied to bank, login and signup3 within the same operation, and the behaviour was verified by performing transactions with the new PIN after the change.

6 Recognising the security weaknesses While testing with unusual inputs, it became clear that queries built by string concatenation can be manipulated through the card number field, and that storing the PIN in plain text offers no protection if the database is read directly. These issues were documented rather than left unnoticed, and correcting them with PreparedStatement and hashed PINs is the first item of future work.

7 Conclusion and Future Enhancements The ATM Simulator System meets all the functional requirements defined for it: a customer can open an account, sign in securely, deposit and withdraw money, use fast cash, view a mini statement, check the balance and change the PIN, with every operation stored permanently in MySQL. The project provided practical

experience of object-oriented design, JDBC programming and the discipline